DESPLIEGUE DE APLICACIONES WEB · 2º DAW

Arquitecturas web

Bloque 2.1 · Fundamentos web: redes, HTTP y arquitecturas



Bienvenidos al restaurante

El hilo conductor: un restaurante

Durante toda la sesión, cada pieza de la web tendrá su equivalente en un restaurante.



Comensal

= Cliente (navegador)

Siempre es quien pide



Cocina

= Servidor

Prepara lo que se pide



Camarero

= Protocolo (HTTP)

Lleva y trae, con sus reglas



Comanda

= Petición

Lo que pides, por escrito



Plato

= Respuesta

Lo que vuelve a la mesa



Dirección del local

= IP y puerto

Dónde está y por qué puerta



Guía de restaurantes

= DNS

Del nombre a la dirección



Número de mesa

= Cookie de sesión

Para que te recuerden

Menú del día

APERITIVO	Redes y modelos: OSI y TCP/IP	20'
ENTRANTE	Cliente-servidor, URL y DNS	15'
PRIMER PLATO	El protocolo HTTP	35'
SEGUNDO PLATO	Estado y sesiones: cookies	15'
POSTRE	Arquitecturas: capas, servicios y código	30'
SOBREMESA	Panorama general y cierre	5'

APERITIVO

Redes y modelos de comunicación

Antes de abrir el restaurante: cómo viaja una comanda



Cómo empezó todo

Red de comunicaciones: dos o más entidades cuya finalidad es intercambiar información.



Como si cada restaurante apuntara las comandas en su propio idioma: ningún camarero podía trabajar en otro local. Hacía falta un estándar.

Modelo, arquitectura y protocolo

Modelo

EN LA WEB

Marco teórico: cómo debe establecerse y producirse la comunicación.

EN EL RESTAURANTE

El manual de cómo funciona cualquier restaurante.

Arquitectura

EN LA WEB

Conjunto de capas y protocolos que implementan la comunicación.

EN EL RESTAURANTE

Cómo se organiza este restaurante en concreto (sala, barra, cocina) y qué normas usa.

Protocolo

EN LA WEB

Reglas que rigen una tarea concreta: HTTP, TCP, IP…

EN EL RESTAURANTE

Las reglas para tomar una comanda y servir un plato.

El modelo OSI: 7 capas

7 Aplicación	Interfaz con las aplicaciones del usuario	*Datos*
6 Presentación	Formato, compresión y cifrado	*Datos*
5 Sesión	Abre y mantiene el diálogo	*Datos*
4 Transporte	Segmenta y garantiza la entrega	*Segmento*
3 Red	Enruta paquetes hasta el destino	*Paquete*
2 Enlace de datos	Tramas entre equipos contiguos	*Trama*
1 Física	Señales, cables y medios	*Bits*



Emisor: los datos bajan de la capa 7 a la 1.

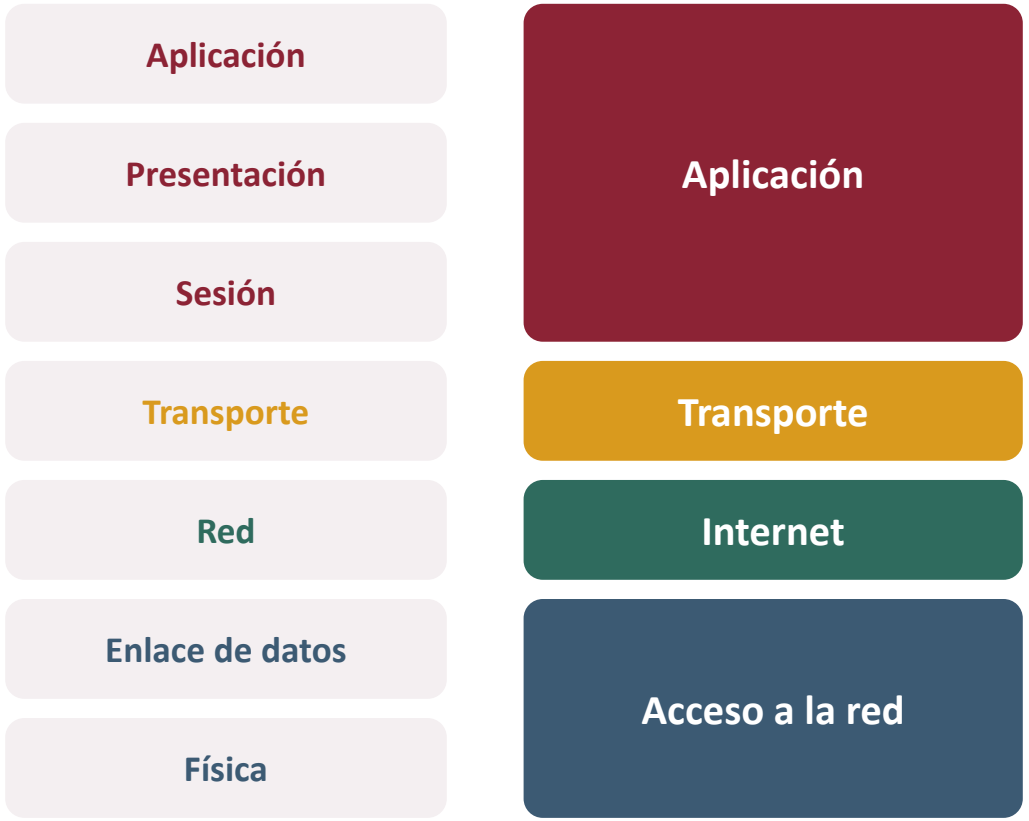


Receptor: suben de la 1 a la 7.

Encapsulación
Cada capa añade su propia información de control; la capa equivalente del receptor la retira.

OSI es un modelo de referencia: sirve para explicar y clasificar.

TCP/IP: el modelo que usa Internet



EN EL RESTAURANTE

Aplicación · HTTP

El camarero escribe la comanda en el idioma de la cocina.

Transporte · TCP

Numera las comandas y comprueba que llegan todas y en orden.

Internet · IP

Les pone la dirección de la cocina y elige el camino.

Acceso a la red · Ethernet, Wi-Fi

El pasillo y la bandeja por donde viajan.

Encapsulación: sobres dentro de sobres



Emisor: mete la comanda en sobres, de dentro a fuera.

Receptor: abre los sobres de fuera a dentro, hasta leer la comanda.

La comanda va en un sobre numerado (TCP), dentro de otro con la dirección de la cocina (IP), sobre la bandeja que la lleva por el pasillo (Ethernet).

Protocolos: la plantilla de camareros

Protocolo	Para qué sirve	Puerto	En el restaurante
HTTP / HTTPS	Páginas web y APIs	80 / 443	*El camarero de sala*
FTP / SFTP	Transferir y gestionar ficheros	21 / 22	*El sumiller: gestiona la bodega*
SMTP · POP3 · IMAP	Enviar y recibir correo	25 · 110 · 143	*El buzón de recados*
DNS	Traducir nombres a direcciones IP	53	*La guía de restaurantes*
DHCP	Dar configuración IP automática	67 / 68	*El maître que te asigna mesa al entrar*
SSH	Acceso remoto seguro	22	*La llave de la puerta de la cocina*
TLS (antes SSL)	Cifrar la comunicación	—	*Hablar en clave con el camarero*



Por debajo de todos: TCP (comandas numeradas, entrega garantizada y en orden) o UDP (gritar el pedido a la barra: rápido, sin confirmar).

¿Me trae un buen vino?



Al camarero de sala (HTTP)

Te lo puede traer sin problema: HTTP descarga ficheros a diario (imágenes, PDF, instaladores…). Para pedir un fichero concreto, va de sobra.



Al sumiller (FTP / SFTP)

Conoce la bodega: además de traer, sube, lista carpetas, renombra y organiza ficheros en el servidor. Es el especialista (UD4).

Cada protocolo está pensado para una tarea. Se puede resolver casi todo «metiéndolo por HTTP», pero no siempre es lo óptimo.

¿Y WebDAV?
Un camarero de sala que se ha formado como sumiller: amplía HTTP con métodos para gestionar ficheros (PROPFIND, MKCOL, COPY…).

IP y puerto: la dirección y la ventanilla

93.184.216.34:443



IP: identifica la máquina. La dirección del restaurante.



Puerto: identifica el programa que escucha. La ventanilla dentro del local.

Un mismo servidor puede tener Nginx en el 443 y MySQL en el 3306: misma dirección, distinta ventanilla.

80 HTTP	**443** HTTPS
21 FTP	22 SSH · SFTP
25 SMTP	53 DNS
3306 MySQL	8080 HTTP alternativo

ENTRANTE

Cliente-servidor, URL y DNS

Llegar al restaurante y hacer el pedido



Cliente-servidor: quién pide y quién sirve



El cliente siempre inicia

El camarero no trae platos que nadie ha pedido.

El servidor es un programa

Espera peticiones en una dirección y un puerto.

Muchos clientes a la vez

Una misma cocina atiende a todas las mesas.

El viaje de una petición, de un vistazo

1

Escribes la URL

Decides a qué restaurante ir y qué pedir

2

DNS: nombre → IP

Buscas la dirección en la guía

3

Conexión TCP a IP:puerto

Llegas y te sientas a la mesa

4

Petición y respuesta HTTP

Pides y te sirven

5

El navegador pinta la página

Disfrutas del plato

En lo que queda de sesión iremos desmontando cada una de estas piezas.

Anatomía de una URL

```
https://www.ejemplo.com:443/productos/42?color=rojo&talla=m#resenas
```

Esquema

Qué protocolo usar

¿Camarero o sumiller?

Host

Dominio o IP del servidor

Qué restaurante

Puerto

Opcional: 80 en HTTP, 443 en HTTPS

Qué ventanilla

Ruta

Qué recurso pides

Qué plato de la carta

Query string

Parámetros clave=valor

«Al punto y sin sal»

Fragmento

Solo lo usa el navegador: nunca llega al servidor

La página de postres de tu carta: no se lo dices al camarero

DNS: la guía de restaurantes

1. **¿Lo tengo en caché?**
   Navegador, sistema operativo y fichero hosts
2. **Pregunto al resolver**
   Normalmente, el DNS de tu red o proveedor
3. **El resolver recorre la jerarquía**
   Raíz → .com → ejemplo.com
4. **Devuelve la IP**
   Y la guarda en caché un tiempo (TTL)
5. **Ya puedo conectar**
   Por HTTP o HTTPS, a esa IP

```
$ nslookup www.ejemplo.com
```



Quieres cenar en «Casa Pepe», pero no sabes dónde está: lo buscas en la guía. La próxima vez ya te sabes la dirección (caché). Si Casa Pepe se muda, tardas un tiempo en enterarte (propagación).

Herramientas: nslookup, dig, host.
Fichero hosts: se consulta antes que el DNS. Útil para «inventar» dominios en local.

PRIMER PLATO

El protocolo HTTP

Conozcamos a nuestro camarero



HTTP: la ficha del camarero



HyperText Transfer Protocol

Nacido en	el CERN, 1989-1991
Creador	Tim Berners-Lee
Forma de trabajar	petición → respuesta
Idioma	texto legible (en HTTP/1.x)
Viaja sobre	TCP, puerto 80
Versión discreta	HTTPS = HTTP + TLS, puerto 443

EN LA WEB

Establece la sintaxis y la semántica del intercambio de documentos entre clientes y servidores web.



HTTPS es el mismo camarero, pero hablando en clave: nadie que escuche desde la mesa de al lado entiende la comanda (UD3).

Hoy, todos contratan a este camarero

HTTP es el idioma común con el que se comunican casi todos los sistemas, a través de APIs.



ERP y CRM

Los subsistemas de la empresa hablan entre sí



Pasarelas de pago

La tienda pide al banco que cobre



Publicidad y afiliación

Anuncios y venta por referencia



Apps móviles

La app consulta a su servidor



El camarero ya no solo atiende la sala: también lleva pedidos a los proveedores, al banco y a los repartidores.

Un servicio completo, paso a paso

	EN LA WEB	EN EL RESTAURANTE
1	Escribes la URL en el navegador	*Eliges restaurante y plato*
2	El navegador separa la URL y consulta el DNS	*Buscas la dirección*
3	Abre una conexión TCP (y TLS si es HTTPS)	*Te sientas; el camarero se acerca*
4	Envía la petición: método, ruta y cabeceras	*Haces la comanda*
5	Responde: código, Content-Type y contenido	*Llega el plato con su etiqueta*
6	Por cada CSS, imagen o JS, una petición más	*Cada guarnición, un viaje más*
7	La conexión se cierra (1.0) o se reutiliza (1.1+)	*El camarero se va o se queda junto a la mesa*

Versiones de HTTP



1991

Solo GET y solo HTML. Sin cabeceras.

1996

Cabeceras, códigos de estado, tipos MIME, POST y HEAD.

1997

Conexiones persistentes y cabecera Host obligatoria.

2015

Formato binario y multiplexación.

2022

Sobre QUIC (UDP) en vez de TCP.

La semántica (métodos, códigos, cabeceras) es la misma desde 1.1. Lo que cambia es cómo viajan los mensajes.

HTTP/2: un camarero que lleva varias comandas a la vez en la misma bandeja.

Métodos: qué le pides al camarero

Método	Para qué sirve	En el restaurante
`GET`	Pedir un recurso	*«Tráigame el plato 42 de la carta»*
`HEAD`	Pedir solo las cabeceras, sin el contenido	*«¿El pescado es de hoy?» (sin que te lo traigan)*
`POST`	Enviar datos para crear o procesar algo	*«Le encargo una tarta para el sábado»*
`PUT`	Reemplazar un recurso completo	*«Cambie mi pedido entero por este otro»*
`PATCH`	Modificar parte de un recurso	*«Mi plato, sin cebolla»*
`DELETE`	Eliminar un recurso	*«Cancele el postre»*
`OPTIONS`	Preguntar qué métodos admite	*«¿Qué se puede pedir a esta hora?»*

Seguro, idempotente… y GET frente a POST



Seguro

No modifica nada en el servidor: solo lee.

Preguntar qué hay de postre.

GET, HEAD, OPTIONS



Idempotente

Repetirlo deja el mismo resultado que hacerlo una vez.

Cancelar el postre dos veces: sigue cancelado. Pedir dos veces una tarta: dos tartas.

GET, HEAD, PUT, DELETE (no POST)

	GET	POST
Los datos van en…	la URL (query string)	el cuerpo de la petición
¿Visibles en historial y logs?	Sí	No directamente
Longitud	Limitada	Prácticamente sin límite
Uso típico	Buscar, leer, filtrar	Formularios, subir ficheros



POST no cifra nada. Sin HTTPS, sus datos viajan igual de legibles que los de GET.

La comanda: el mensaje de petición

```
1  POST /registro HTTP/1.1
2  Host: www.miweb.com
   User-Agent: curl/8.5.0
   Content-Type: application/x-www-form-urlencoded
   Content-Length: 18
3  ↵  (CRLF)
4  nombre=Ana&edad=20
```

1 Línea de petición

Método, ruta y versión.

Qué plato, de qué carta.

2 Cabeceras

Información extra: Clave: Valor.

Las notas: mesa 7, sin gluten…

3 Línea en blanco

Marca el fin de las cabeceras (CRLF).

Fin de las notas.

4 Cuerpo (opcional)

Los datos enviados: vacío en GET.

Lo que entregas: el formulario.

El plato: el mensaje de respuesta

```
1  HTTP/1.1 200 OK
2  Date: Wed, 18 Sep 2019 19:44:53 GMT
   Server: Apache/2.4.38 (Debian)
   Content-Type: text/html; charset=UTF-8
   Content-Length: 144
3  ↵  (CRLF)
4  <!DOCTYPE html><html>…
```

1 Línea de estado

Versión, código y frase.

«Aquí tiene»: ha ido bien.

2 Cabeceras

Fecha, servidor, tipo y tamaño.

La etiqueta del plato.

3 Línea en blanco

Fin de las cabeceras.

Fin de la etiqueta.

4 Cuerpo

El recurso pedido: HTML, imagen, JSON…

El plato en sí.

Códigos de estado: cómo ha ido el servicio

1xx	2xx	3xx	4xx	5xx
Informativo	**Éxito**	**Redirección**	**Error del cliente**	**Error del servidor**
«Un momento, enseguida le atiendo»	*«Aquí tiene»*	*«Eso se sirve en otra sala»*	*«Eso no se lo puedo servir: revise su pedido»*	*«Lo sentimos, problema en cocina»*

La primera cifra lo dice todo: con ver un 4xx ya sabes que el problema está en la petición; con un 5xx, en el servidor.

Los códigos que hay que saber

200

OK

«Aquí tiene su plato»

201

Created

«Su reserva está creada»

301

Moved Permanently

«Nos hemos mudado a la calle Mayor»

302

Found

«Hoy servimos en la terraza»

304

Not Modified

«Su pan sigue siendo de hoy: no le traigo otro»

401

Unauthorized

«¿A nombre de quién está la reserva?»

403

Forbidden

«La zona VIP no es para usted»

404

Not Found

«Ese plato no está en la carta»

500

Internal Server Error

«Se ha quemado la cocina»

503

Service Unavailable

«Cocina saturada: vuelva más tarde»

401, 403 y 404: no los confundas

401 — Falta autenticación

¿Quién eres? No te has identificado, o lo has hecho mal.

«¿Tiene reserva? Dígame su nombre.»

Va acompañado de WWW-Authenticate

403 — Falta autorización

Sé quién eres, pero no tienes permiso para esto.

«Ya sé quién es usted, pero la cocina no es para clientes.»

Identificarse otra vez no sirve

404 — No existe

En esa URL no hay ningún recurso.

«Ese plato no está en la carta.»

Revisa la ruta

¿Y el 500? El pedido era correcto, pero la cocina ha fallado. No es culpa del cliente.

Cabeceras: las notas de la comanda

Pares **`Clave: Valor`** que acompañan a cada petición y respuesta con metadatos.

```
Host: www.miweb.com
Accept-Language: es-ES
Cookie: SESSIONID=a8f3k29xq
```



«Mesa 7 · sin gluten · la carta en español · cliente habitual nº 42»

Generales

En peticiones y respuestas

`Date, Cache-Control`

En cualquier nota

De petición

Las envía el cliente

`Host, User-Agent, Accept, Cookie`

Las escribe el comensal

De respuesta

Las envía el servidor

`Server, Location, Set-Cookie`

Las escribe la cocina

De entidad (contenido)

Describen el cuerpo

`Content-Type, Content-Length`

La etiqueta del plato

Cabeceras que vas a ver siempre

`Host`

Obligatoria en HTTP/1.1. Permite que una sola IP sirva muchos dominios.

Una sola cocina que cocina para varias marcas de comida a domicilio.

`User-Agent`

Quién hace la petición: navegador, versión, sistema operativo.

Lo dice el propio cliente: no te fíes.

`Accept-*`

Formatos, idiomas y compresiones que acepta el cliente.

«La carta en español, por favor».

`Content-Type`

Tipo MIME del cuerpo: text/html, application/json…

La etiqueta: «esto es un tinto».

`Location`

A dónde ir: acompaña a las redirecciones y al 201.

«La terraza está por allí».

`Set-Cookie / Cookie`

El servidor entrega una cookie y el cliente la devuelve.

«Su número de mesa es el 12» (segundo plato).

Content-Type: la etiqueta del plato

Tipo MIME	Contenido
`text/html`	Página HTML
`text/css`	Hoja de estilos
`text/javascript`	Código JavaScript
`application/json`	Datos JSON (APIs)
`image/png` · `image/jpeg` · `image/svg+xml`	Imágenes
`application/x-www-form-urlencoded`	Formulario HTML enviado por POST
`multipart/form-data`	Formulario con ficheros adjuntos



Sin etiqueta, la cocina no sabe si le has dado un vino o un vinagre. Content-Type le dice al otro lado cómo interpretar el cuerpo.

Suele llevar el juego de caracteres:
`text/html; charset=UTF-8`

SEGUNDO PLATO

Estado y sesiones

El camarero con amnesia



HTTP no recuerda nada



Comensal

«Soy Ana, la de la mesa 12. Ya puede traerme el segundo.»



Camarero

«¿Qué Ana? ¿Qué mesa? ¿Qué segundo?»

HTTP es «sin estado» (stateless). Cada petición es independiente: el servidor no recuerda, por sí mismo, las peticiones anteriores.



El problema

¿Cómo sabe el servidor que sigues siendo tú: que ya iniciaste sesión, qué hay en tu carrito…?

Cookies: el ticket con tu número



EN LA WEB

1. **`Set-Cookie`**: el servidor entrega la cookie.
2. **`Cookie`**: el navegador la guarda y la devuelve sola.
3. El servidor te reconoce en cada petición.

Te dan un ticket con un número. Lo enseñas cada vez que pides, y así te reconocen.

Sesiones: el estado vive en la cocina

Lo que lleva el cliente

`SESSIONID=a8f3k29xq`

Solo la llave: un ID aleatorio.

Lo que guarda el servidor

`a8f3k29xq`	usuario: ana · carrito: 3 productos · rol: cliente
`7b2fq91mz`	usuario: luis · carrito: vacío · rol: admin
`c41xk00pa`	usuario: marta · carrito: 1 producto · rol: cliente

En memoria, en ficheros o en una base de datos.

Tu ticket solo dice «mesa 12». Lo que has pedido, lo que llevas gastado y si eres cliente habitual está apuntado en la libreta de la cocina.

La cookie no guarda tus datos: guarda el número para encontrarlos.

El lado oscuro de las cookies



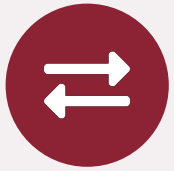
Robo de sesión

EN LA WEB

Un atacante lee tu cookie (por ejemplo, con un script inyectado: XSS) y se hace pasar por ti sin conocer tu contraseña.

EN EL RESTAURANTE

Alguien copia tu ticket y pide a tu nombre.



CSRF

EN LA WEB

El navegador envía tus cookies aunque la petición la haya iniciado otra web. Un atacante puede ejecutar acciones en tu nombre.

EN EL RESTAURANTE

Otro local hace un pedido en tu nombre, y tu ticket se enseña solo.



Seguimiento

EN LA WEB

Cookies de terceros que siguen tu navegación de una web a otra.

EN EL RESTAURANTE

Alguien apunta en qué restaurantes cenas cada noche.

Defensas básicas (se amplían en la UD3)

```
Set-Cookie: SESSIONID=a8f3k29xq; Path=/; HttpOnly; Secure; SameSite=Lax
```

	EN LA WEB	EN EL RESTAURANTE
HttpOnly	JavaScript no puede leer la cookie: frena el robo por XSS.	*El ticket va en un bolsillo que solo abre el camarero.*
Secure	La cookie solo viaja por HTTPS, nunca en claro.	*Solo enseñas el ticket dentro del local, nunca por la calle.*
SameSite	No se envía en peticiones iniciadas desde otras webs: mitiga CSRF.	*El ticket solo vale si lo enseñas tú, en este restaurante.*

Son una capa extra de defensa: no sustituyen a una aplicación bien hecha.

POSTRE

Arquitecturas web

Cómo se organiza el restaurante por dentro



De 1 a 3 capas

1 capa



Food truck

Una sola persona toma nota, cocina y cobra.

Interfaz, lógica y datos juntos, en la misma máquina. Scripts pequeños.

2 capas



Bar pequeño

Barra por un lado; cocina y almacén por otro.

Cliente (interfaz) + servidor (lógica y datos mezclados).

3 capas



Restaurante

Sala, cocina y despensa, cada una con su equipo.

Presentación, negocio y datos separados.

Arquitectura de 3 capas



Presentación · *La sala*

Lo que ve el usuario: HTML, CSS, JS o una app móvil.



Negocio · *La cocina*

Reglas, validaciones y cálculos.



Datos · *La despensa*

Acceso y persistencia: base de datos, ficheros.

Cambiar una sin tocar las demás

Redecoras la sala sin vaciar la despensa.

Escalar cada una por separado

Más camareros los sábados, sin ampliar la despensa.

Repartir el trabajo en equipo

Cada equipo en su zona.

N capas: la misma idea con capas intermedias (presentación → API → negocio → acceso a datos → BD). 3 capas es el caso más común.

Monolito frente a microservicios



Monolito

Un gran restaurante con una sola cocina que lo hace todo. Se abre, se reforma y se amplía entero.



Microservicios

Un mercado gastronómico: puestos independientes (pizza, sushi, postres), cada uno con su cocina y su despensa.

	Monolito	Microservicios
Despliegue	Una sola unidad	Independiente por servicio
Complejidad inicial	Baja	Alta: hay que coordinar servicios
Escalado	Toda la aplicación a la vez	Servicio a servicio
Fallos	Un fallo puede afectar a todo	Un fallo puede quedar aislado
Proyecto pequeño	Buena opción	Puede ser sobreingeniería

Otros estilos de organización



SOA

Servicios grandes que se comunican a través de un bus central (ESB). Típico de grandes empresas.

Una cadena hotelera con una central de compras común.



Orientada a eventos

Los componentes publican eventos («pedido creado») y reacciona quien le interese, sin conocerse.

La campana: «¡mesa 5 lista!», y acude quien le toca.



Serverless / FaaS

Despliegas funciones; la nube decide dónde y cuándo ejecutarlas. Pagas por ejecución.

Un cocinero por horas: pagas solo cuando cocina y no gestionas la cocina.

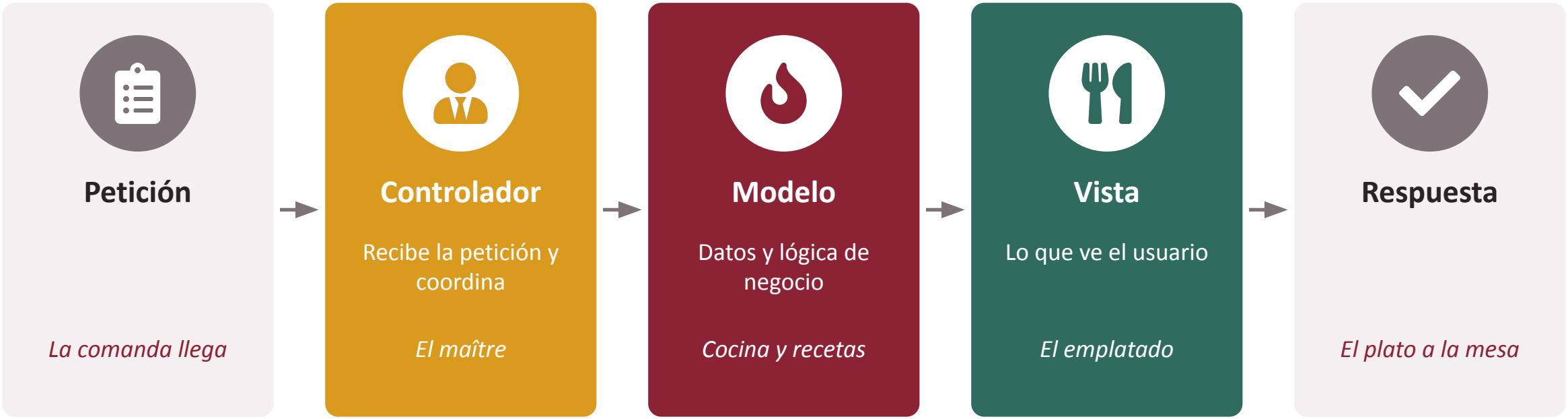


Peer-to-peer (P2P)

Sin servidor central: cada nodo es cliente y servidor a la vez (BitTorrent).

Una cena de traje: cada uno trae un plato y todos comen de todos.

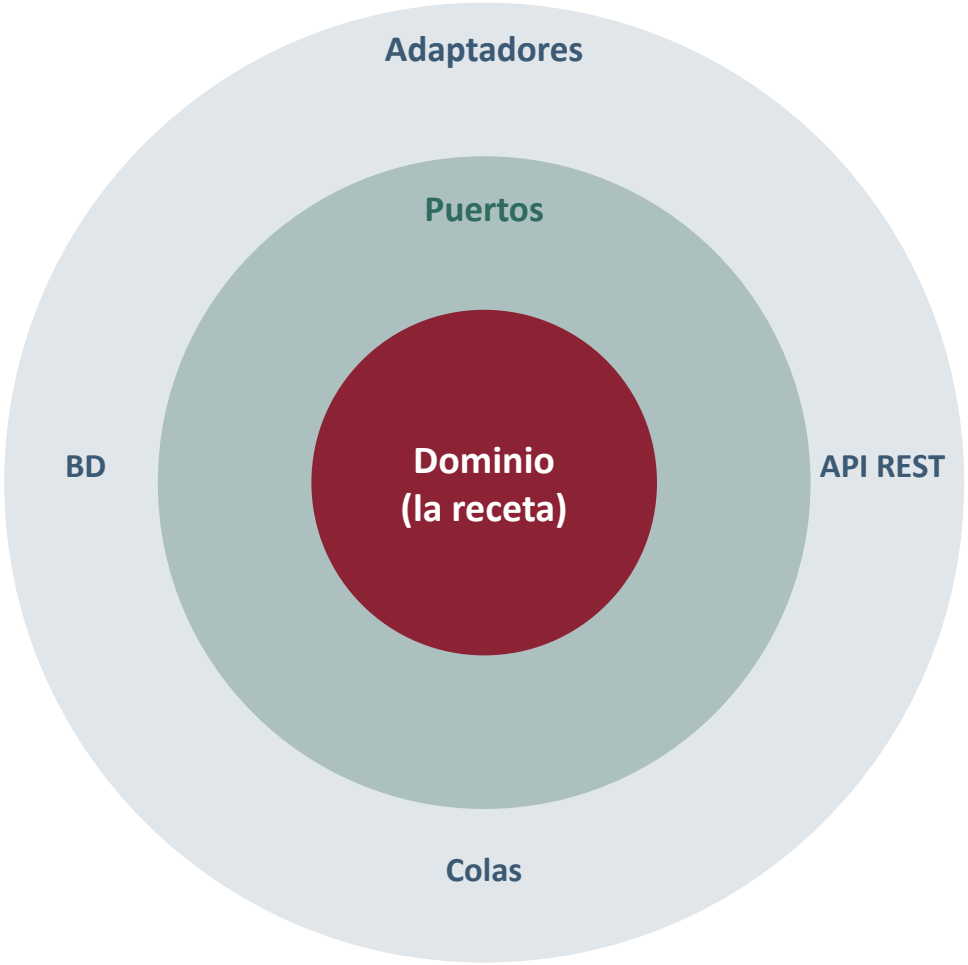
MVC: quién hace qué dentro de la cocina



El patrón más usado en frameworks web: Spring MVC, Django, Ruby on Rails, Laravel…

Variantes: MVP y MVVM (Vue, Angular). Mismo objetivo: separar qué se ve de cómo se calcula.

Hexagonal, Clean y Onion: la receta en el centro



Dominio en el centro

La lógica de negocio, aislada de los detalles técnicos.

Puertos y adaptadores

Interfaces (puertos) e implementaciones concretas (adaptadores): BD, API REST, colas…

Las dependencias apuntan hacia dentro

El dominio nunca depende de un framework ni de una BD.

Cambias de proveedor de verduras (o de base de datos) sin cambiar la receta. Y puedes probar la receta sin ir al mercado.

SOBREMESA

Panorama general

Repasamos la cena completa



Dos preguntas, dos familias



¿Cómo reparto el sistema en piezas desplegables?

Monolito · SOA · Microservicios · Orientada a eventos · Serverless · P2P

¿Un gran restaurante o un mercado gastronómico?



¿Cómo organizo el código de cada pieza?

Capas (1, 2, 3, N) · MVC, MVP, MVVM · Hexagonal, Clean, Onion

¿Cómo se organiza cada cocina por dentro?

No compiten: se combinan. Un microservicio puede estar organizado por dentro en 3 capas con arquitectura hexagonal.

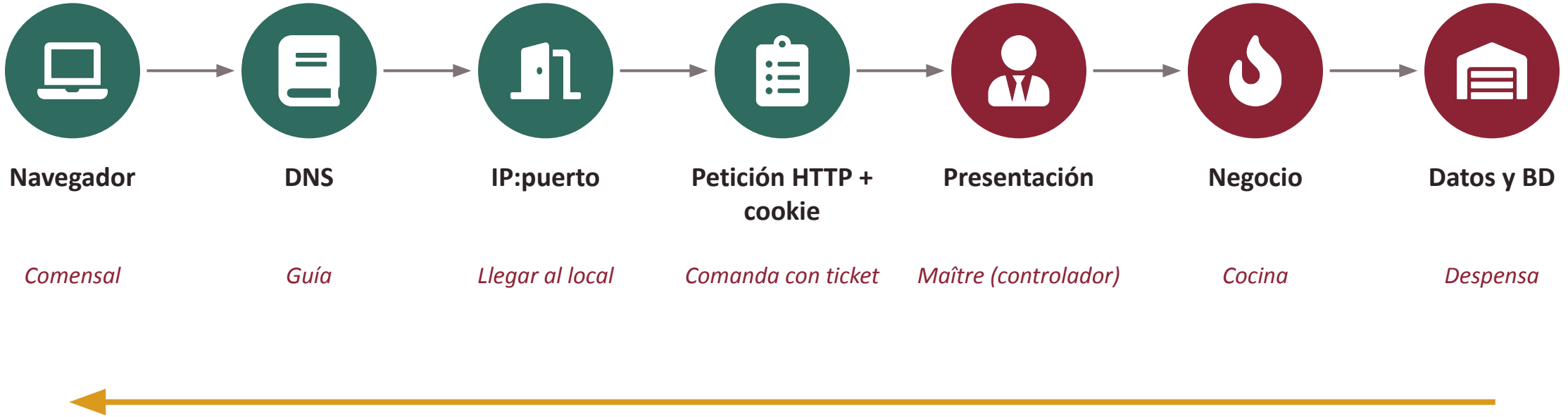
¿Cuándo uso cada una?

Escenario	Arquitectura razonable
Proyecto pequeño, equipo reducido, primera versión	**Monolito en 3 capas**
Sistema grande, muchos equipos, despliegues independientes	**Microservicios**
Integrar sistemas heredados de una gran empresa	**SOA**
Picos de carga impredecibles, tareas puntuales	**Serverless / FaaS**
Componentes que deben reaccionar a cambios sin acoplarse	**Orientada a eventos**
Lógica de negocio compleja que debe sobrevivir a cambios de framework	**Hexagonal / Clean**



No montes un mercado gastronómico para servir cuatro mesas.

La cena completa: el viaje de una petición



← **Respuesta HTTP:** código de estado + cabeceras (Content-Type, Set-Cookie…) + cuerpo. *El plato vuelve a la mesa y el navegador pide sus guarniciones (CSS, JS, imágenes).*

La cuenta, por favor: ideas clave

HTTP vive en la capa de aplicación y viaja sobre TCP/IP. Un servicio se localiza por IP + puerto.

Petición: método, ruta, versión, cabeceras, línea en blanco y cuerpo opcional.

2xx éxito · 3xx redirección · 4xx error del cliente · 5xx error del servidor. 401 ≠ 403.

Las cabeceras son metadatos: Host, Content-Type, Location, Set-Cookie…

HTTP no tiene estado: la cookie lleva un ID; el estado de la sesión vive en el servidor.

Capas y patrones organizan el código; monolito o microservicios reparten el sistema. Se combinan.

PARA LLEVAR

Ahora, a la cocina



Práctica: HTTP, cookies y sesiones con curl.

Próximo día: dónde vive la cocina (servidores web y de aplicaciones).